Same File, Different Answers: Execution-Equivalent Settings Change Benchmark Results of Quantized LLMs on CPUs

Zayd Mulani
Independent researcher
zaydmulani@gmail.com

November 2026

Abstract

People compare quantized language models by running benchmarks locally, usually with llama.cpp on a CPU, and treat a difference of a point or two as a property of the weights. I measure how much of such a difference can come from settings that do not change the model at all: the CPU backend tier the runtime picks for the machine, the micro-batch size, flash attention, weight repacking, and the server's prompt cache and parallel slots. With the release, the GGUF file, the prompts and greedy decoding held fixed, I ran six model files through a teacher-forced grid of twelve such settings and 980 benchmark items (MMLU, TriviaQA, GSM8K) under a subset of them. A rerun of the same configuration is bitwise identical and thread count never matters. Most other settings do. For Qwen3-0.6B in Q4_K_M, loading the AVX tier instead of the AVX2 tier changed 41 of 600 MMLU answers and 20% of all generated outputs while no task's accuracy moved by more than 1.3 points, and two runs of the identical parallel-server command agreed bitwise on 45% of items. A pre-registered test traces the size of the effect to activation quantization: the same tier change leaves an F32 model at the measurement floor but gives a mean KL divergence of about 10^{-3} for Q8_0 and 10^{-2} for Q4_K_M, because rounding activations to 8-bit integers turns float32 rounding differences into full quantization steps. Measured against the gap between quantization formats, the execution-only change is about 60% of the Q8_0-versus-F16 divergence. I give a short protocol for pinning these settings and for measuring the noise floor before comparing two quantized files. Code, raw per-item outputs and the pre-registration are public.

1 Introduction

In October 2026 I ran a pre-registered gate for a quantization study on a small language model: a candidate format passed only if its MMLU score stayed at or above 51.0 on 200 questions. It scored 50.5 and failed by one question. The same report noted that running another format twice, with identical code, had given 107 and 106 correct MMLU answers, because the evaluation used a four-slot `llama-server` whose batches form differently on every run. The gate stayed failed as written, but it left a question I could not answer at the time: how much of a measured difference between two quantized models, run locally on a CPU, comes from the models and how much from how the arithmetic happened to be executed?

This paper measures that. I hold the release, the weights file, the prompts and greedy decoding fixed and change only settings that the documentation presents as performance options with no

effect on the model: the CPU backend tier the runtime loads, the thread count, the micro-batch size, flash attention, weight repacking, the server's prompt cache and its number of parallel slots. I call these *execution-equivalent* configurations. In exact arithmetic they compute the same function. In float32 on a CPU, most of them do not, and the differences are large enough to change answers.

The setting is llama.cpp on x86 CPUs, the common way to run quantized GGUF models on consumer hardware. Two properties make it a good case. First, one prebuilt release ships several compiled copies of its CPU kernels and picks one at start-up from the machine's instruction-set flags, so two people running the same command on different laptops silently run different code. Second, quantized matrix products round activations to 8-bit integers before every multiplication. Prior work on inference nondeterminism concentrates on GPU serving stacks [5, 9, 10, 19, 20]; this paper looks at the CPU path, where neither batch invariance nor cross-machine agreement has been measured as far as I can find.

Findings. On Qwen3-0.6B in Q4_K_M, with 980 items from MMLU, TriviaQA and GSM8K:

- Running the same configuration twice is bitwise reproducible, and thread count never mattered in any test. Neither did the `haswell` tier or micro-batches of 8 and 64. Every other setting changed the logits for at least one file, and the AVX and SSE tiers, micro-batch 1 and flash attention changed them for all six.
- Running the same file on the AVX-only backend tier instead of the AVX2 tier left no item bitwise identical, changed 20% of generated outputs and 41 of 600 MMLU answers, and produced 37 correct/incorrect flips, while every task's accuracy moved by at most 1.3 points. On wikitext the tier change alone gives a mean KL divergence of 0.011 from the reference and changes the top-1 token at 6% of positions.
- The common parallel evaluation setup (four slots, four concurrent clients, prompt cache on) is not reproducible from run to run: two runs of the identical command agreed bitwise on only 45% of items and produced different GSM8K reasoning chains for 47 of 80 problems.
- The size of these effects is explained by activation quantization. With F32 weights and an F32 KV cache, the same tier change leaves the logits within the measurement floor and changes no top-1 token; with Q8_0 weights the mean KL divergence is above 10^{-3}, and with Q4_K_M above 10^{-2}. Rounding activations to integers turns ulp-level float differences into full-step integer differences that propagate through the network.

Contributions. (1) A measurement of how much execution-equivalent settings change outputs and benchmark scores for quantized models on CPUs, across six model files and up to twelve settings, with all raw per-item outputs released. (2) A pre-registered test that identifies activation quantization as the amplifier, plus code-level explanations, with pre-registered predictions, of which settings are safe and which are not. (3) A short protocol for local evaluations: which settings to pin, what to report, and how to size the noise floor before comparing two quantized models. Section 2 describes the runtime, Section 4 the design, Section 5 the measurements, Section 6 the mechanism and Section 7 the protocol.

2 Background: how one release runs one file

This section describes the parts of the llama.cpp CPU backend that matter for the experiments. Everything here refers to release b11696 (commit `ceefd16da`, 16 October 2026); file names are relative to `ggml/src/ggml-cpu/`.

One release, several kernels sets. The prebuilt Windows CPU package of a llama.cpp release ships one `ggml-cpu-*.dll` per x86 microarchitecture tier (`x64`, `sse42`, `sandybridge`, `ivybridge`, `haswell`, `alderlake`, `skylakex`, `icelake`, `zen4` and others). At start-up the loader scores each library against the CPU's feature flags and loads the best one. The user does not choose and is not told, apart from a `system_info` log line. Two people who download the same release and the same GGUF file therefore run different compiled kernels if their CPUs differ. Because the selection only looks at which libraries are present, a single machine can run any tier its CPU supports by leaving only that library in the folder. This is how the ISA experiments in this paper emulate older or simpler hardware on one laptop.

Where the ISA enters the arithmetic. Quantized matrix products split into an integer part and a floating-point part. For a Q4_0 weight block and a Q8_0 activation block, the 32 products of 4-bit and 8-bit integers are summed in integer arithmetic, which is exact on every instruction set. The block result is then scaled by the two block scales and accumulated across the row in single precision. That float accumulation is where tiers differ. In `arch/x86/quants.c` the AVX2 branch of `ggml_vec_dot_q4_0_q8_0` accumulates with `_mm256_fmadd_ps`, a fused multiply-add with one rounding; the AVX branch uses `_mm256_add_ps(_mm256_mul_ps(...))`, which rounds twice; the SSSE3 and generic branches use other lane layouts and summation orders. The same split appears in the Q4_K, Q6_K and F16 dot products and in the vector helpers used by normalization and softmax (`simd-mappings.h` defines `GGML_F32x8_FMA` as `_mm256_fmadd_ps` only when `__FMA__` is defined).

Weight repacking. On CPUs that report AVX2, Q4_0 and Q4_K weight matrices are copied at load time into an interleaved 8×8 layout (`repack.cpp`, `ggml_repack_get_optimal_repack_type`); Q8_0, Q6_K and F16 are not repacked on x86. The repacked matmul processes the activation rows of a micro-batch in groups of four with a `gemm` kernel and handles the remaining n mod 4 rows one at a time with a `gemv` kernel (`forward_mul_mat_one_chunk`). The two kernels accumulate in different orders. A token's logits can therefore depend on its row index within its micro-batch modulo four, which is a function of prompt length, micro-batch size, prompt-cache hits and which other requests share the batch.

Prompt path versus generation path. For multi-row products llama.cpp calls `llamafile_sgemm` (tinyBLAS) when the shape qualifies: never for a single row (`n < 2`), and for Q8_0 and Q4_0 only when $n \geq 8$ or $n = 4$; F16 additionally needs F16C. Otherwise it falls back to the per-row `vec_dot`. Attention's $QK^\top$ product reads the F16 key cache and switches path with the number of query rows in the same way. The llama.cpp server documentation already warns that logits are not guaranteed to be bit-identical between prompt processing and token generation, and that the prompt cache can therefore make results nondeterministic.

Knobs a user treats as neutral. The following settings are documented as performance or memory options and do not change the model: the thread count (`-t`), the logical and physical batch sizes (`-b`, `-ub`), flash attention (`-fa`), weight repacking (`-repack`), the number of server slots (`-np`) and the prompt cache (`cache_prompt`, on by default in the server). In exact arithmetic every one of them computes the same function. I call two configurations *execution-equivalent* when they differ only in such settings or in the selected ISA tier, and use the same release, the same GGUF file, the same prompt and greedy decoding.

3 Related work

Floating-point non-associativity. That reordering a floating-point sum changes its value, and that fused multiply-add changes rounding, is textbook material [8, 12, 16]. In deep learning it has been studied mostly as a source of training variance across runs, libraries and hardware [17, 21] and of run-to-run variance in GPU kernels [18]. None of these isolates the inference-time effect of a runtime that picks among compiled kernel variants per machine.

Nondeterminism in LLM inference. Atil et al. [1] report accuracy swings of up to 15% across repeated runs of API models set to be deterministic, without access to the serving stack. Yuan et al. [19] show on GPUs that precision, GPU count and type, and batch size change reasoning-model outputs, and propose storing weights in 16 bits while computing in FP32. He [10] attribute most user-visible serving nondeterminism to a lack of batch invariance in kernels rather than to concurrency, and describe batch-invariant RMSNorm, matmul and attention. A line of 2026 systems work reduces the cost of determinism on GPUs: verified speculation [9], padding with isolated prefill [20], shape-determined reduction orders that agree across GPU generations [5], and power-of-two INT8 scales that make two GEMM libraries agree bitwise [3]. All of this targets data-centre GPUs. Zhao et al. [20] build on the observation that most GPU kernels are position-invariant even when they are not batch-invariant; Section 6 shows that the CPU repack kernels studied here are not position-invariant.

Variance and flips in evaluation. Madaan et al. [14] quantify seed variance across benchmarks, Miller [15] gives statistical tools for comparing models on evals, and Biderman et al. [2] document practical reproducibility problems in the lm-evaluation-harness. Dutta et al. [6] show that compressed models can match a baseline's accuracy while many individual answers flip, and propose flip rate and KL divergence as complements to accuracy. The present paper asks a narrower question that these do not separate: how many flips appear when nothing about the model changes, only the way the same quantized file is executed on a CPU.

Local inference. llama.cpp and its GGUF format are the common path for running quantized models on consumer CPUs [7]. Quantization studies compare formats by perplexity, KL divergence to the full-precision model and benchmark accuracy, usually from one machine and one configuration. I found no measurement of how much of a reported difference could come from the execution configuration of a single file.

Table 1: Model files. Hashes are the first 16 hex digits of SHA-256.

label	file	weight types	SHA-256 prefix
q06-f16	Qwen3-0.6B F16	F16	`b166332e0224681e`
q06-q80	Qwen3-0.6B Q8_0	Q8_0	`f34d7b4d1035d121`
q06-q4km	Qwen3-0.6B Q4_K_M	Q4_K, Q6_K	`683e1b8a8ed85096`
q06-q40	Qwen3-0.6B Q4_0	Q4_0	`34a119e136d15e41`
g3-1b-q40	Gemma-3-1B-it QAT Q4_0	Q4_0	`6364f1139062ff56`
q35-08-q40	Qwen3.5-0.8B Q4_0	Q4_0 (hybrid attention / linear recurrence)	`f9a8630a4a96076b`

4 Method

4.1 Setup

All runs use one laptop: an Intel Core i7-1355U (2 performance and 8 efficiency cores, 12 hardware threads; AVX2, FMA, F16C and AVX-VNNI, no AVX-512) with 12 GB of RAM, running Windows 11 (build 26300). The software is the unmodified prebuilt CPU package of llama.cpp release b11696 (`llama-b11696-bin-win-cpu-x64.zip`, built with Clang 20.1.8). For each ISA tier I made a folder containing the release with exactly one `ggml-cpu-*.dll`; the start-up log confirms which tier loaded. Six tiers load on this CPU: `x64`, `sse42`, `sandybridge` (AVX), `ivybridge` (AVX, F16C), `haswell` (AVX2, FMA, F16C) and `alderlake` (`haswell` plus AVX-VNNI). The AVX-512 tiers refuse to load. `alderlake` is the tier the loader picks on this machine when all libraries are present.

Models. Six GGUF files cover four quantization types and three architectures (Table 1). Q8_0 and Q4_K_M were produced from the F16 file with the release's `llama-quantize`. As a control, quantizing on the `sse42` tier, on the `alderlake` tier, and with 1 or 8 threads gave byte-identical files (SHA-256 below), so the weights themselves do not depend on the machine that made them.

Configurations. The reference configuration R is: `alderlake` tier, 8 threads for both generation and batch processing, logical batch 2048, micro-batch (`-ub`) 512, flash attention off, repacking on, one server slot, one client request at a time, prompt cache off. Every other configuration changes exactly one factor (Table 2). `server4` is the setting a parallel evaluation client typically uses against `llama-server`: four slots, four concurrent requests and the server's default prompt cache. I ran it twice with identical commands (`server4-a`, `server4-b`) to measure run-to-run variation, and ran R twice for the same purpose.

The two kinds of measurement below cost very different amounts of CPU time. A teacher-forced run takes about a minute; a benchmark run takes 40 to 55 minutes. So the teacher-forced grid covers every setting for every model, and the benchmark runs cover a subset chosen in advance for the primary model (Qwen3-0.6B Q4_K_M) and only R, one tier change and `server4` for the others. I dropped benchmark runs whose teacher-forced result already showed bitwise agreement with R (`haswell`, thread counts) and, as the CPU budget ran short, runs that the grid showed to be numerically equivalent to one already queued. The full list of runs and the reason for each omission are in `harness/jobs.py`.

Table 2: Execution-equivalent configurations. Each row differs from the reference in one factor. The last two columns give the models it was run for in the teacher-forced grid and in the benchmark runs.

name	change from the reference	KL grid	benchmark runs
ref-rerun	none (second run)	all	Q4_K_M
isa-haswell, -sse42	CPU backend tier	all	none
isa-ivybridge	tier: AVX, F16C, no FMA	all	F16
isa-sandybridge	tier: AVX, no F16C, no FMA	all	Q4_K_M, Qwen3 Q4_0
t8, t12	8 or 12 threads (KL reference: 4)	all	none
ub1, ub8	micro-batch 1 or 8	all	none
ub7, ub64	micro-batch 7 or 64	all	Q4_K_M
fa-on	flash attention on	all	Q4_K_M
repack-off	weight repacking off	all	none
cache-on	server prompt cache on	none	Q4_K_M
server4-a	4 slots, 4 clients, prompt cache on	none	all six
server4-b	second run of server4-a	none	Q4_K_M

4.2 Evaluation tasks

I reuse a frozen evaluation set built earlier for an unrelated quantization study with fixed seeds: zero-shot MMLU [11], 4-shot chain-of-thought GSM8K [4] and 5-shot TriviaQA [13]. Each configuration runs the first 600 MMLU, 300 TriviaQA and 80 GSM8K items of that file (980 items). Prompts are raw completions with no chat template. Decoding is greedy (temperature 0, top-k 1). MMLU is scored by reading the top-20 next-token log-probabilities and taking the highest-scoring answer letter; its *margin* is the log-probability gap between the best and second-best letter. TriviaQA generates up to 24 tokens and is scored by normalized alias match; GSM8K generates up to 256 tokens and is scored by the last number after `####`.

4.3 Measurements

For every item the harness records the generated token ids, the log-probability of each chosen token and the first step's top-20 list, exactly as the server prints them (shortest round-trip decimal of a float32). Against the reference run of the same model I report:

- **identical**: token ids, every chosen-token log-probability and the first-step top-20 list are equal. Equality of these float strings implies equality of the underlying float32 values.
- **same output**: the generated token ids are equal, so the visible answer is equal.
- **flip**: the item is scored correct in one run and wrong in the other. I report flips in both directions, the accuracy change, and an exact McNemar test on the discordant pairs.
- For generative tasks, the **first divergence**: the index of the first generated token that differs.

Separately, a teacher-forced grid uses `llama-perplexity` with `-kl-divergence` on 12 chunks of 512 tokens of WikiText; the tool scores the second half of each chunk, so each run covers 3,072 positions. Its reference is `alderlake` with 4 threads (so that it could share the CPU with the benchmark queue), otherwise as R. For each model it saves the reference logits once and then reports, for each configuration, the mean and maximum KL divergence from the reference and the fraction of positions whose top-1 token agrees. The tool stores the reference log-probabilities

Table 3: Teacher-forced divergence from the reference configuration on 12 chunks of 512 WikiText tokens. Each cell is mean KLD / top-1 agreement (%). "=" marks a setting that is indistinguishable from the reference at the tool's storage precision (mean |KLD| $\leq 10^{-6}$ and 100% top-1 agreement). UBSIXFOUR Reference: `alderlake`, 4 threads, micro-batch 512, flash attention off, repack on.

configuration	q06-q4km	q06-q40	q06-q80	q06-f16	g3-1b-q40	q35-08-q40
ref-rerun	=	=	=	=	=	=
isa-haswell	=	=	=	=	=	=
isa-ivybridge	1.2e-2 / 94.05	2.4e-3 / 97.19	1.3e-3 / 98.37	2.0e-6 / 99.94	1.4e-3 / 97.94	1.3e-3 / 98.01
isa-sandybridge	1.1e-2 / 94.31	2.5e-3 / 97.19	1.4e-3 / 98.14	2.0e-6 / 99.90	1.3e-3 / 98.20	1.3e-3 / 97.91
isa-sse42	1.1e-2 / 93.76	2.6e-3 / 97.29	1.4e-3 / 98.04	2.0e-6 / 99.90	1.4e-3 / 97.97	1.3e-3 / 97.94
t8	=	=	=	=	=	–
t12	=	=	=	=	=	–
ub1	1.2e-2 / 94.58	2.4e-3 / 97.52	1.3e-3 / 98.11	2.0e-6 / 99.97	1.3e-3 / 98.07	–
ub7	1.2e-2 / 93.86	=	=	=	=	–
ub8	=	=	=	=	=	–
ub64	=	=	=	=	=	–
fa-on	1.1e-2 / 94.38	2.5e-3 / 97.61	1.4e-3 / 97.97	1.0e-6 / 99.94	1.4e-3 / 98.46	–
repack-off	1.1e-2 / 94.51	2.5e-3 / 97.48	=	=	1.3e-3 / 98.01	–

in a compressed 16-bit form, so two bitwise-identical runs do not read exactly zero: they show |mean KLD| $\leq 10^{-6}$ and a maximum around 6×10^{-5}. I call this the storage floor. A configuration at the floor with 100% top-1 agreement is reported as indistinguishable from the reference; exact bit equality is established only by the benchmark runs.

4.4 Pre-registration

Before reading any result I wrote down predictions derived from the source code (`PREDICTIONS.md`, committed at the start of the run; summarized in Section 6). The design is one-factor-at-a-time; I did not search for configurations that maximize differences.

5 Results

5.1 Which settings change the logits

Table 3 is the teacher-forced map: for each model file and each execution-equivalent setting, the mean KL divergence of the next-token distribution from the reference and the fraction of positions whose top-1 token is unchanged.

Two groups appear. Rerunning the reference, changing the thread count, and the `haswell` tier are indistinguishable from the reference for every model; so are micro-batches of 8 and 64. Every other setting differs from the reference for at least one model, and the differences within a model are of one size: for Q4_K_M every differing setting gives a mean KLD between 0.011 and 0.012 and changes the top-1 token at 5.4 to 6.2% of positions, whether the change is the backend tier, micro-batch 1, flash attention or turning repacking off. The size depends on the weight format, not on which setting caused the difference: about 10^{-2} for Q4_K_M, 2.5×10^{-3} for Qwen3 Q4_0, 1.3×10^{-3} for Q8_0, Gemma Q4_0 and Qwen3.5 Q4_0, and 2×10^{-6} for F16, where only one to three of 3,072 top-1 tokens change. Section 6 explains this pattern.

Which settings differ also depends on the format. Turning repacking off changes nothing for Q8_0 and F16, which are not repacked on x86, and micro-batch 7 changes only Q4_K_M.

Table 4: Benchmark runs of Qwen3-0.6B Q4_K_M against the reference run R (980 items: 600 MMLU, 300 TriviaQA, 80 GSM8K). "ident.": items whose tokens and log-probabilities are bitwise equal to R. "same out": items whose generated tokens are equal. Flips are over all 980 items, reference right and other wrong / reference wrong and other right. p: exact McNemar test on the flips.

configuration	ident. %	same out %	MMLU ans. changed	flips	p	ΔMMLU	ΔTQA	ΔGSM
ref-rerun	100.0	100.0	0	0 (0/0)	1.00	+0.0	+0.0	+0.0
sandybridge tier	0.0	79.7	41	37 (15/22)	0.32	+0.5	+1.0	+1.3
micro-batch 7	0.0	80.2	49	47 (23/24)	1.00	+0.2	-0.3	+1.3
prompt cache on	31.7	91.2	22	23 (14/9)	0.40	-0.5	-0.7	+0.0
server4 (run a)	11.6	85.6	21	32 (13/19)	0.38	+0.3	+0.7	+2.5
server4 (run b)	11.5	85.0	20	25 (9/16)	0.23	+0.0	+1.7	+2.5
server4 a vs b	45.4	87.9	13	17 (8/9)	1.00	-0.3	+1.0	+0.0

5.2 What the differences do to benchmark answers

Teacher-forced divergence is a property of the logits; what matters to a benchmark user is whether answers change. Table 4 summarizes the benchmark runs for the primary model (the full table for all models is Table 8 in the appendix).

A second run of R reproduced all 980 items bit for bit, so the reference is deterministic and every difference below comes from the one changed setting. Loading the AVX tier (`sandybridge`) instead of `alderlake` left no item bitwise identical: every first-step log-probability moved (median change of the top token's log-probability 0.10 nats). One generated output in five differed, 41 of 600 MMLU answers changed and 37 items flipped between right and wrong (15 one way, 22 the other). Micro-batch 7 did the same (49 MMLU answers changed, 47 flips). Yet every accuracy moved by at most 1.3 points, and no McNemar test was close to significance ($p \geq 0.32$). The flips cancel because they are not caused by the model getting better or worse; they are coin tosses on items the model was unsure about.

That is visible in the reference margins (Figure 1, left). Across the five benchmark settings that differ from R, 71 distinct MMLU items changed their answer at least once. Their median reference margin was 0.28 nats, against 1.65 for all items. Of the 44 items with a margin under 0.25 nats, 33 changed under at least one setting; of the 424 items with a margin of at least 1 nat, one did. The largest margin that changed was 1.25 nats.

On GSM8K the divergence happens early and then compounds (Figure 1, right). Under the AVX tier 52 of 80 reasoning chains differ from the reference, with a median first differing token at position 8; after the first different token the two runs write different solutions. The prompt cache is the exception: it changed 22 MMLU answers but none of the 80 GSM8K outputs. I did not trace why. One candidate is that the shared few-shot prefix keeps every remaining prompt row at the same position modulo 4 in its micro-batch, which F4 in `PREDICTIONS.md` says is what matters for the Q4_K kernels.

The effect is smaller for the other formats in the same order as the teacher-forced grid. Under the AVX tier, Qwen3 Q4_0 changed 23 MMLU answers (24 flips, 85.8% same outputs). The F16 file under `ivybridge`, the tier that removes fused multiply-add, kept 99.3% of outputs and changed 2 MMLU answers, although no item was bitwise identical.

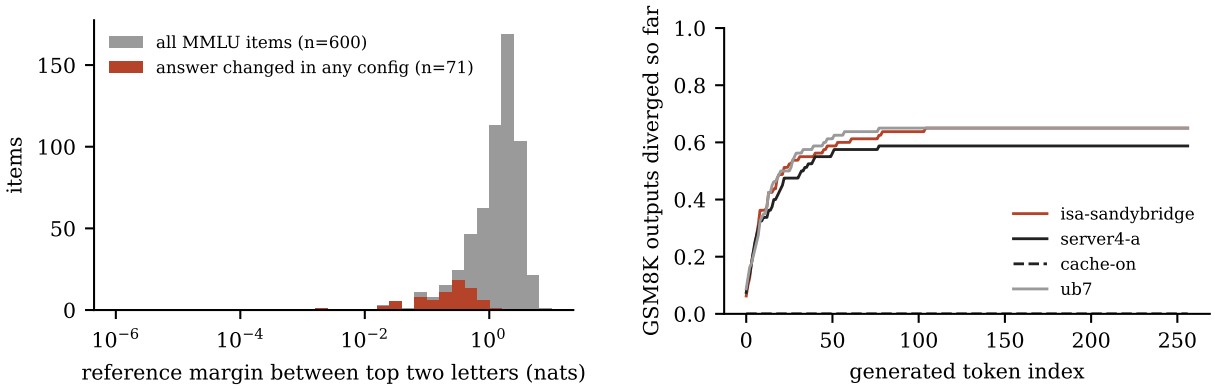


Figure 1: Left: reference answer margins (log-probability gap between the two best letters) for all MMLU items and for the items whose answer changed under at least one setting (Q4_K_M). Changed answers come from the low-margin tail. Right: fraction of GSM8K outputs that have diverged from the reference by each generated token index, per setting. The prompt-cache curve lies at zero.

5.3 Run-to-run variation under a parallel server

The `server4` setting is how a parallel evaluation harness usually drives `llama-server`. Its numbers depend on request timing: how many requests share a decode step and how prompts split across micro-batches change from run to run. For Q4_K_M, each of the two runs agreed bitwise with R on fewer than 12% of items, and the two runs agreed with each other bitwise on 45.4% (445 of 980). They gave the same visible output on 87.9% of items, and 33 of 80 GSM8K chains; 17 items flipped between the two runs and GSM8K accuracy happened to match. A user who runs this command twice and compares answers sees two different models.

For the other files the parallel server mattered less, again following the format: Qwen3 Q4_0 kept 71.6% of items bitwise identical to R and Q8_0 73.5%, with 6 and 3 flips. GEMMASERVER

5.4 How large is this relative to a quantization comparison?

To judge whether these numbers matter, Table 5 compares the four Qwen3-0.6B formats with each other, all at R. Going from F16 to Q8_0 changes 32 MMLU answers and 40 items flip; going from F16 to Q4_K_M changes 173 answers and costs 8.7 MMLU points. The AVX tier change on the Q4_K_M file, with no change to the weights, changed 41 MMLU answers, more than the whole F16-to-Q8_0 quantization step did. In teacher-forced terms the tier change on Q8_0 (mean KLD 0.0013) is about 60% of the divergence between Q8_0 and F16 (0.0023), 11% of the Q4_K_M-to-F16 divergence and 1.3% of the Q4_0-to-F16 divergence (`quantkl.py`).

So execution noise does not hide large quantization losses: the 8-point drop from F16 to Q4_K_M is far outside the accuracy spread of any execution change here. It does make small comparisons unreliable. A one- or two-point difference between two formats on a benchmark of a few hundred items is within what a change of backend tier or server setting produces on one file, and an answer-level comparison (which items each format gets right) is dominated by execution noise for the closest pairs.

Table 5: Differences between quantization formats of the same model, all run at R. Compare with the execution-only differences in Table 8.

pair (both at R)	same out %	MMLU ans. changed	flips	ΔMMLU	ΔGSM
q06-q4km vs q06-q80	44.8	158	147	+8.2	+20.0
q06-q40 vs q06-q4km	26.0	121	123	-0.2	-7.5
q06-q40 vs q06-q80	32.4	184	166	+8.0	+12.5
q06-f16 vs q06-q4km	44.0	173	161	-8.7	-18.8
q06-f16 vs q06-q40	31.7	204	186	-8.5	-11.2
q06-f16 vs q06-q80	87.7	32	40	-0.5	+1.3

6 Why small rounding differences become different answers

Section 2 explains why execution-equivalent configurations produce different float32 results: different kernels sum in different orders, and only some tiers fuse multiply and add. That alone does not explain the size of the effect. Float32 reordering perturbs a result by roughly 10^{-7} relative per operation, yet Table 3 shows mean KL divergences of 10^{-3} to 10^{-2} and top-1 disagreement on several percent of positions. This section tests one explanation and checks the code-level predictions.

6.1 Activation quantization amplifies rounding differences

Before every quantized matrix product, llama.cpp converts the float32 activation row into 8-bit integers per block: blocks of 32 values for Q8_0 (used with Q4_0 and Q8_0 weights) and of 256 values for Q8_K (used with K-quants). Each value is mapped to $\mathrm{round}(127\,x/\max|x_{\mathrm{block}}|)$. A perturbation of one unit in the last place almost never matters, but when it moves a value across a rounding boundary, the stored integer changes by one, which is an error of $\max|x|/127$, several orders of magnitude larger than the perturbation that caused it. A small simulation (`amp_sim.py`) makes the scale concrete: perturbing Gaussian activations with sparse outliers by one ulp of relative error changes only 4.8×10^{-6} of the Q8_0 codes, but those changes raise the relative error of the quantized row from 1.2×10^{-7} to 2.9×10^{-5}, a factor of about 250, in a single matmul. A transformer applies this step seven times per layer, and the errors feed forward.

Weights stored as F16 also quantize activations, but to F16 (11 significant bits instead of about 8, and no shared block scale), and F32 weights with an F32 KV cache do not discretize activations at all. If amplification is the main mechanism, the same execution change should produce far smaller divergence for those formats. I pre-registered this test after one probe (Addendum A in `PREDICTIONS.md`) and ran it on Qwen3-0.6B in six formats (Table 6).

The prediction held. With nothing discretized (F32 weights, F32 KV cache), removing FMA (`ivybridge`) or SIMD (`sse42`) changes no top-1 token in 3,072 positions and the KL divergence is at the measurement floor. Storing the KV cache in F16 or the weights in F16 changes one to three positions. Switching to Q8_0 weights, with everything else equal, raises the mean KLD from below 10^{-6} to 1.3×10^{-3} and the top-1 disagreement to about 2%. Q4_0 is about twice Q8_0, and Q4_K_M, whose activations use 256-value Q8_K blocks, is about ten times Q8_0. I did not isolate why Q4_K_M is so much more sensitive; the larger activation block (a coarser step for most values in the block) is one candidate. The thread-count control is exactly zero in every format.

The same amplification explains why the AVX-only tiers differ so much from the AVX2 tiers even though their integer arithmetic is identical: any float32 difference upstream of an activation quantizer, whether from FMA, a different summation order or a different kernel, is converted into

Table 6: Divergence from the `alderlake` reference for the same model in six formats, under two ISA changes and a thread-count change (8 chunks of 512 tokens). "$\leq 10^{-6}$" is the storage floor of the measurement: configurations that are bitwise identical also read $|\text{KLD}| \leq 10^{-6}$. F32+f32KV also stores the KV cache in F32.

format	ivybridge		sse42		t4	
	mean KLD	top-1 %	mean KLD	top-1 %	mean KLD	top-1 %
F32+f32KV	$\leq 10^{-6}$	100.00	$\leq 10^{-6}$	100.00	$\leq 10^{-6}$	100.00
F32	$\leq 10^{-6}$	99.95	$\leq 10^{-6}$	99.95	$\leq 10^{-6}$	100.00
F16	$\leq 10^{-6}$	99.90	$\leq 10^{-6}$	99.90	$\leq 10^{-6}$	100.00
Q8_0	0.0013	98.23	0.0014	98.04	$\leq 10^{-6}$	100.00
Q4_K_M	0.0121	93.82	0.0118	93.68	$\leq 10^{-6}$	100.00
Q4_0	0.0024	97.11	0.0026	97.16	$\leq 10^{-6}$	100.00

integer differences of a full step at the next layer.

6.2 Code-level predictions

Table 7 lists the predictions written before the runs and their outcomes.

Of the thirteen predictions the runs tested, twelve held. The clear failure is micro-batch 7, which I expected to differ from micro-batch 512 for every model because the repacked kernels process the rows of a micro-batch in groups of four with one routine and the remainder with another (F4 in `PREDICTIONS.md`). It differs only for Q4_K_M. For Q4_0 the grouped and remainder routines are generated from one template and evidently produce the same bits per row, while the Q4_K routines are separately written kernels that do not. For Q8_0, which is not repacked, micro-batch 7 also moves the matrix products from the tinyBLAS path to the per-row dot product path (F5), yet the logits are identical; both paths accumulate each 32-value block with one fused multiply-add into an 8-lane float32 accumulator and reduce it at the end, which is consistent with the match, though I did not check them instruction by instruction. Micro-batch 1 differs everywhere, so the attention path, which changes with the number of query rows, is the likely source of that difference. The general lesson is that whether a performance knob changes the numbers depends on the weight format, so a setting found safe for one file is not safe for another.

7 A protocol for local evaluations

None of the effects above is a bug in the usual sense: each kernel computes a correct float32 approximation, and the documentation warns about prompt-cache nondeterminism. The problem is that evaluation practice treats these settings as irrelevant. The following steps cost little and make local results comparable.

1. **Report the execution configuration with the score.** Release tag, the CPU backend tier that loaded (the `system_info` line names its features), micro-batch size, flash attention, repack, number of slots, prompt cache, and the KV cache type. Thread count can be omitted; it never changed a result here.

2. **Pin what can be pinned.** For comparisons between models, run both on the same machine with the same backend library, one slot, sequential requests and the prompt cache off. This makes runs bitwise reproducible (Section 5) at the cost of throughput.

Table 7: Pre-registered predictions (`PREDICTIONS.md`) and outcomes.

prediction	outcome	evidence
P1 threads 4/8/12 identical	supported	6/6 models (Table 3)
P2 `haswell` = `alderlake`	supported	6/6 models
P3 `ivybridge` differs	supported	6/6 models
P4 `sandybridge` differs from `ivybridge`; `sse42` differs from AVX tiers	not tested	both differ from R; the tiers were never compared with each other
P5 micro-batch 1 differs	supported	6/6 models
P5 micro-batch 7 differs	**falsified** for 5/6	differs only for Q4_K_M; identical for Q4_0 (3 models), Q8_0, F16
P6 repack off differs for Q4_0, Q4_K_M only	supported	differs for all four Q4 files, identical for Q8_0 and F16
P7 flash attention differs	supported	6/6 models
P8 rerun of R identical	supported	980/980 items bitwise equal
P9 prompt cache changes some items	supported	68% of items not bitwise equal (Q4_K_M)
P10 two identical parallel-server runs differ	supported	Q4_K_M only (the only model run twice)
P11 flips concentrate on low-margin items	supported	Figure 1
A1 F32+f32KV < F16 < Q8_0, ≥10×	supported	Table 6
A2 thread control exactly zero	supported	all six formats

3. **Measure the noise floor before comparing.** Run the reference model under two or three execution-equivalent configurations (for example a second tier and a different micro-batch size) and count flips. Any model-versus-model difference that is not clearly larger than the spread among these runs should not be reported as a difference. For the 980-item set used here, accuracy moved by at most 2.5 points on the 80-item GSM8K subset (two items) and 0.5 points on 600 MMLU items, but up to 49 MMLU answers changed. The accuracy spread understates the noise; the answer-level spread is the better measure.

4. **Use paired statistics.** Report discordant pairs (flips in each direction) and an exact McNemar test rather than two accuracies with independent error bars [15]. Item-level agreement between two configurations of the same file is a direct estimate of how much of a model comparison's discordance is execution noise.

5. **Do not gate on single questions.** A pre-registered threshold should include a margin at least as large as the measured execution spread, or be evaluated under several configurations.

For runtime developers, the amplification result suggests where determinism is cheapest to buy. Thread count is already invariant. Making the activation quantizer's input invariant to the kernel path (for example by fixing the reduction order of the float epilogue that feeds it) would remove most of the divergence between tiers, and making the repacked matmul use one kernel for every row regardless of n mod 4 would remove the dependence on batch composition.

8 Limitations

One machine, emulated tiers. All ISA tiers ran on one Raptor Lake laptop by restricting which backend library could load. A real Sandy Bridge or SSE4.2-only machine runs the same

compiled code, so the arithmetic should match, but I did not verify this on such hardware. AVX-512 tiers (`skylakex`, `icelake`, `zen4`), ARM CPUs (where Q4_0 repacks to different layouts) and GPU backends were not tested; each adds kernel variants, so the map in Table 3 is a lower bound on the number of distinct numerical behaviours one release can show.

Small models, small subsets. The models have 0.6 to 1 billion parameters and the evaluation uses 980 items per configuration. Small models have flatter output distributions and so more low-margin items, which likely inflates flip rates relative to larger models. Absolute flip counts should not be read as estimates for 7B or 70B models; the mechanism and the existence of the effect do not depend on model size.

Release-specific mechanism. The kernel facts in Section 2 hold for release b11696. llama.cpp changes quickly, and the repack thresholds, tinyBLAS conditions and tier list may differ in other releases. The measurement harness is release-agnostic and can be rerun.

One-factor design. Each configuration differs from the reference in one factor. Interactions (for example a different tier and the parallel server together, which is what two different users would actually run) were not measured and would add, not remove, variation.

References

[1] Berk Atil, Sarp Aykent, Alexa Chittams, et al. Non-determinism of "deterministic" LLM settings. arXiv:2408.04667, 2024.

[2] Stella Biderman, Hailey Schoelkopf, Lintang Sutawika, et al. Lessons from the trenches on reproducible evaluation of language models. arXiv:2405.14782, 2024.

[3] Teng-Ruei Chen. Deterministic LLM inference across GPU kernels: power-of-two INT8 quantization scales and the limits of tolerance-based conformance. arXiv:2609.00363, 2026.

[4] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, et al. Training verifiers to solve math word problems. arXiv:2110.14168, 2021.

[5] Liam Cooper, Shinnung Jeong, Hyeran Jeon, et al. Accelerating the mitigation of LLM inference nondeterminism across GPU architectures. arXiv:2609.25624, 2026.

[6] Abhinav Dutta, Sanjeev Krishnan, Nipun Kwatra, and Ramachandran Ramjee. Accuracy is not all you need. arXiv:2407.09141, 2024.

[7] Georgi Gerganov and contributors. llama.cpp: LLM inference in C/C++. `https://github.com/ggml-org/llama.cpp`, release b11696, 2026.

[8] David Goldberg. What every computer scientist should know about floating-point arithmetic. *ACM Computing Surveys*, 23(1):5–48, 1991.

[9] Raja Gond, Aditya K. Kamath, Ramachandran Ramjee, and Ashish Panwar. LLM-42: enabling determinism in LLM inference with verified speculation. arXiv:2601.17768, 2026.

[10] Horace He and Thinking Machines Lab. Defeating nondeterminism in LLM inference. Thinking Machines Lab: Connectionism, September 2025. `https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/`.

[11] Dan Hendrycks, Collin Burns, Steven Basart, et al. Measuring massive multitask language understanding. In *ICLR*, 2021.

[12] IEEE. IEEE standard for floating-point arithmetic (IEEE Std 754-2019), 2019.

[13] Mandar Joshi, Eunsol Choi, Daniel Weld, and Luke Zettlemoyer. TriviaQA: a large scale distantly supervised challenge dataset for reading comprehension. In *ACL*, 2017.

[14] Lovish Madaan, Aaditya K. Singh, Rylan Schaeffer, et al. Quantifying variance in evaluation benchmarks. arXiv:2406.10229, 2024.

[15] Evan Miller. Adding error bars to evals: a statistical approach to language model evaluations. arXiv:2411.00640, 2024.

[16] David Monniaux. The pitfalls of verifying floating-point computations. *ACM Transactions on Programming Languages and Systems*, 30(3):1–41, 2008.

[17] Hung Viet Pham, Shangshu Qian, Jiannan Wang, et al. Problems and opportunities in training deep learning software systems: an analysis of variance. In *ASE*, 2020.

[18] Sanjif Shanmugavelu, Mathieu Taillefumier, Christopher Culver, Oscar Hernandez, Mark Coletti, and Ada Sedova. Impacts of floating-point non-associativity on reproducibility for HPC and deep learning applications. arXiv:2408.05148, 2024.

[19] Jiayi Yuan, Hao Li, Xinheng Ding, et al. Understanding and mitigating numerical sources of nondeterminism in LLM inference. arXiv:2506.09501, 2025.

[20] Shiju Zhao, Jiacheng Yang, Qihang Chen, et al. CoRun: padding is simple and efficient for deterministic LLM inference. arXiv:2608.14376, 2026.

[21] Donglin Zhuang, Xingyao Zhang, Shuaiwen Leon Song, and Sara Hooker. Randomness in neural network training: characterizing the impact of tooling. In *MLSys*, 2022.

A Reproduction

All code, the frozen evaluation set, every per-item output and the pre-registration file are in the accompanying repository. The steps below reproduce the results on any x86-64 Windows machine with AVX2; other CPUs will load a different set of tiers.

1. Download `llama-b11696-bin-win-cpu-x64.zip` from the llama.cpp releases page and make one folder per tier, each holding the whole release except all `ggml-cpu-*.dll` files but one. Run any tool once per folder and check the `system_info` line.
2. Obtain the GGUF files in Table 1 (Q8_0, Q4_K_M and F32 are produced from the F16 file with `llama-quantize`) and check the hashes.
3. `py -3 harness/jobs.py 0 1` runs the evaluation queue; `py -3 harness/kl.py 12` runs the teacher-forced grid; `py -3 harness/amp.py 8` runs the amplification test; `py -3 harness/analyze.py` and `py -3 harness/figs.py` rebuild every table and figure in this paper from the raw outputs.

Each evaluation run writes one JSON line per item with the generated token ids, the float32 log-probability of every chosen token and the first step's top-20 list, so any comparison in this paper can be recomputed without rerunning a model.

B Full evaluation table

Table 8: Every evaluation run against its model's reference run R. "ident.": items whose tokens and log-probabilities are bitwise equal; "same out": items whose generated tokens are equal; "MMLU ans. changed": of 600; flips are over all 980 items (reference right and other wrong / reference wrong and other right). The last row per model compares the two `server4` runs with each other.

model	configuration	ident. %	same out %	MMLU ans. changed	flips (r→w / w→r)	ΔMMLU pts	ΔTQA pts	ΔGSM pts
q06-q4km	ref-rerun	100.0	100.0	0	0 (0/0)	+0.0	+0.0	+0.0
	sandybridge tier	0.0	79.7	41	37 (15/22)	+0.5	+1.0	+1.3
	micro-batch 7	0.0	80.2	49	47 (23/24)	+0.2	-0.3	+1.3
	prompt cache on	31.7	91.2	22	23 (14/9)	-0.5	-0.7	+0.0
	server4 (run a)	11.6	85.6	21	32 (13/19)	+0.3	+0.7	+2.5
	server4 (run b)	11.5	85.0	20	25 (9/16)	+0.0	+1.7	+2.5
	server4 a vs b	45.4	87.9	13	17 (8/9)	-0.3	+1.0	+0.0
q06-q40	sandybridge tier	0.0	85.8	23	24 (12/12)	-0.2	+0.0	+1.3
	server4 (run a)	71.6	94.8	2	6 (3/3)	+0.2	-0.3	+0.0
q06-q80	server4 (run a)	73.5	97.7	0	3 (2/1)	+0.0	+0.0	-1.3
q06-f16	ivybridge tier	0.0	99.3	2	2 (2/0)	-0.2	+0.0	-1.3